

The next step is to merge these two sorted half lists into one sorted full list named `sortedResult`. We'll maintain two indices for the `left` and `right` lists named `iLeft` and `iRight`. Inside a loop, the smaller of the two values ④ is appended to `sortedResult`, and its respective index variable (either `iLeft` or `iRight`) is incremented. If either `iLeft` or `iRight` reaches the end of its list, the remaining items in the other half's list are appended to `sortedResult`.

Let's follow an example of the merging step if the recursive calls have returned `[2, 9]` for `left` and `[5, 8]` for `right`. Since these lists were returned from `mergeSort()` calls, we can always assume they are sorted. We must merge them into a single sorted list in `sortedResult` for the current `mergeSort()` call to return to its caller.

The `iLeft` and `iRight` indices begin at 0. We compare the value at `left[iLeft]` (which is 2) and `right[iRight]` (which is 5) to find the smaller one:

```
sortedResult = []
    left: [2, 9]    right: [5, 8]
    indices:  0  1          0  1
    iLeft = 0      ^
    iRight = 0      ^
```

Since `left[iLeft]`'s value, 2, is the smaller of the values, we append it to `sortedResult` and increase `iLeft` from 0 to 1. The state of the variables is now as follows:

```
sortedResult = [2]
    left: [2, 9]    right: [5, 8]
    indices:  0  1          0  1
    iLeft = 1      ^
    iRight = 0      ^
```

Comparing `left[iLeft]` and `right[iRight]` again, we find that of 9 and 5, `right[iRight]`'s 5 is smaller. The code appends the 5 to `sortedResult` and increases `iRight` from 0 to 1. The state of the variables is now the following:

```
sortedResult = [2, 5]
    left: [2, 9]    right: [5, 8]
    indices:  0  1          0  1
    iLeft = 1      ^
    iRight = 1      ^
```
